```
# Practical 1: Text Pre-processing
# Stop word removal and stemming

import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

# Download required NLTK data
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

# Input text
text = "Information retrieval systems are used to retrieve useful information from documents."

# Convert text into words
words = word_tokenize(text.lower())

# Get English stop words
stop_words = set(stopwords.words("english"))

# Remove stop words
filtered_words = []

for word in words:
    if word.isalpha() and word not in stop_words:
        filtered_words.append(word)

# Create stemmer
stemmer = PorterStemmer()

# Apply stemming
stemmed_words = []

for word in filtered_words:
    stemmed_words.append(stemmer.stem(word))

# Display results
print("Original Text:")
print(text)

print("\nAfter Stop Word Removal:")
print(filtered_words)

print("\nAfter Stemming:")
print(stemmed_words)
```

Show hidden output

```
# Practical 2: Inverted Index

# Documents
documents = {
    1: "The quick brown fox jumped over the lazy dog",
    2: "The lazy dog slept in the sun",
    3: "The fox is a clever animal"
}

# Create an empty inverted index
inverted_index = {}

# Build the inverted index
for doc_id, text in documents.items():

    # Convert text into words
    words = text.lower().split()

    for word in words:

        # Remove simple punctuation
        word = word.strip(".,!?")

        # If word is not present, create a new list
        if word not in inverted_index:
            inverted_index[word] = []

        # Add document ID if not already present
        if doc_id not in inverted_index[word]:
            inverted_index[word].append(doc_id)
```

```
# Display inverted index
print("Inverted Index:\n")

for word, doc_list in inverted_index.items():
    print(word, "->", doc_list)

# Search for a word
query = input("\nEnter a word to search: ").lower()

if query in inverted_index:
    print("Word found in documents:", inverted_index[query])
else:
    print("Word not found.")
```

Show hidden output

```
# Practical 3: Bayesian Network for Heart Disease

!pip install pgmpy


# Import required libraries
import pandas as pd

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination

# Read the heart disease dataset
data = pd.read_csv("heart.csv")

# Select the required columns
data = data[["cp", "exang", "slope", "Heartdisease"]]

# Create the Bayesian Network
model = DiscreteBayesianNetwork([
    ("cp", "Heartdisease"),
    ("exang", "Heartdisease"),
    ("slope", "Heartdisease")
])

# Learn probabilities from the dataset
model.fit(data, estimator=MaximumLikelihoodEstimator)

# Create inference object
inference = VariableElimination(model)

# Give patient information
result = inference.query(
    variables=["Heartdisease"],
    evidence={
        "cp": 4,
        "exang": 1,
        "slope": 2
    }
)

# Display diagnosis probabilities
print(result)
```

Show hidden output

Next steps: Explain error

```
# Practical 4: Agglomerative Hierarchical Clustering

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Read the credit card dataset
data = pd.read_csv("CC GENERAL.csv")

# Select two numerical columns
X = data[["BALANCE", "PURCHASES"]]
```

```
# Remove missing values
X = X.dropna()

# Create the dendrogram
linked = linkage(X, method="ward")

plt.figure(figsize=(8, 5))

dendrogram(linked)

plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")

plt.show()

# Apply Agglomerative Clustering
model = AgglomerativeClustering(
    n_clusters=3,
    linkage="ward"
)

# Create clusters
clusters = model.fit_predict(X)

# Display cluster numbers
print("Cluster labels:")
print(clusters)
```

```
# Practical 5: PageRank Algorithm

import networkx as nx

# Create a directed graph
graph = nx.DiGraph()

# Add links between pages
graph.add_edges_from([
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("C", "A"),
    ("D", "A"),
    ("D", "C")
])

# Calculate PageRank
pagerank = nx.pagerank(graph)

# Display PageRank values
print("PageRank values:\n")

for page, score in pagerank.items():
    print(page, ":", round(score, 4))
```

```
PageRank values:

A : 0.3797
B : 0.1989
C : 0.3839
D : 0.0375
```

```
Start coding or generate with AI.
```